

Appendix: Prompt Design Explanation and Iteration

Overview

The assignment requires that each LLM step must have clearly designed prompts where the output of one step directly becomes the input of the next step, along with at least one example of prompt iteration after testing. This appendix explains the full prompt design process used for the League of Legends Dynamic Matchup Strategist agent and documents how prompts were improved after observing real failures during testing.

The system uses four major LLM calls:

1. Step 1 — Input Parsing
2. Step 3 — Matchup Analysis
3. Step 4 — Tactical Timeline Generation
4. Step 5 — Final Markdown Report Formatting

Each prompt was designed to solve a single cognitive task instead of combining multiple responsibilities into one request. This improves reliability, traceability, and debugging.

Step 1 — Input Parsing

Purpose

The first LLM call converts messy natural language user input into structured JSON that the rest of the pipeline can use. Since tool calls require structured parameters, this step must happen first.

Example input:

“I’m playing Lux bot with Yuumi support against Caitlyn and Zyra. Their jungler is Lee Sin.”

Expected output:

```
{  
  "user_role": "ADC",  
  "user_champ": "Lux",  
  "damage_type": "AP",  
  "lane_opponent": "Caitlyn",  
  "ally_support": "Yuumi",
```

```
"enemy_support": "Zyra",  
"ally_jungle": null,  
"enemy_jungle": "Lee Sin"  
}
```

Why this prompt was separated

If parsing is mixed with strategy generation, the model often skips details or invents missing information. By forcing strict JSON first, later steps become deterministic and tool queries become reliable.

Prompt Iteration Example 1 — The Lux Bot Lane Failure

Initial Failure

During testing, when the user entered:

“Lux bot lane with Yuumi support”

the LLM correctly labeled the role as ADC, but Step 3 later assumed Lux should receive traditional AD marksman items such as Essence Reaper, Doran’s Blade, and Berserker’s Greaves.

This happened because the model treated “bot lane” and “ADC” as the same thing and ignored the fact that Lux is fundamentally an AP mage.

Change Made

The Step 1 prompt was updated to explicitly extract:

```
"damage_type": "AP | AD | MIXED"
```

and included role rules such as:

“If a champion that is typically AP (Lux, Ziggs, Xerath, Brand, Seraphine) is described as playing bot lane or ADC, still list user_role as ADC but set damage_type to AP.”

Why It Works

This prevents downstream reasoning from depending only on role stereotypes. Instead of assuming all ADC roles require AD builds, later prompts use explicit categorical information.

This was one of the most important improvements in the entire system.

Step 3 — Matchup Analysis

Purpose

This is the core reasoning step of the project.

It combines:

- structured parsed champion data from Step 1
- live web search results from Step 2

and produces:

- power spikes
- item builds
- matchup difficulty
- win conditions
- build reasoning

This step must be separate because strategic analysis is fundamentally different from formatting or timeline generation.

Prompt Iteration Example 2 — Build Hallucinations

Initial Failure

The model frequently produced weak or incorrect builds such as:

- Jinx receiving Essence Reaper instead of hyper-carry crit builds
- Lux receiving AD marksman builds
- Magic Resist being recommended against Zed (a pure AD assassin)
- Removed items like Galeforce being hallucinated

This happened because DuckDuckGo snippets were sometimes too short to show complete builds, so the model fell back to weak training assumptions.

Change Made

A strict “CRITICAL RULES FOR BUILD” section was added.

Example:

- Do not suggest removed, obsolete, or past-season items
- Match items to the champion’s real kit scaling

- Traditional ADC champions must receive attack speed + crit scaling builds
- AP champions in carry roles must receive high-damage AP itemization
- Defensive items must correctly counter enemy damage type
- Never suggest Magic Resist against pure AD assassins like Zed

Additional AP/AD build gates were also added:

Example:

Lux (AP):

- Doran's Ring
- Sorcerer's Shoes
- Luden's / Shadowflame / Rabadon's / Zhonya's

Jinx (AD):

- Doran's Blade
- Berserker's Greaves
- Kraken / Infinity Edge / Runaan's

Why It Works

Instead of trusting the model to infer modern meta builds automatically, the prompt explicitly constrains valid reasoning paths.

This dramatically improved strategic correctness.

Step 4 — Tactical Timeline Generation

Purpose

This step converts static matchup analysis into a minute-by-minute ranked game plan from minute 1 to minute 20.

This includes:

- wave management
- trade patterns
- jungle tracking
- vision placement
- objective control

- recall timing
- rotations
- mid-game macro transitions

This step is separated because temporal planning is different from static matchup reasoning.

Prompt Iteration Example 3 — Repetitive Timelines

Initial Failure

The model generated highly repetitive outputs such as:

Minute 12: Farm safely
Minute 13: Farm safely
Minute 14: Continue farming
Minute 15: Maintain pressure

It also ignored:

- Dragon timing
- Rift Herald
- turret plates
- lane rotations
- unknown jungle pressure

When enemy jungle was null, it even produced:

“Jungle Threat: None”

which is strategically incorrect.

Change Made

A strict “CRITICAL RULES FOR TIMELINE” section was added.

Key rules included:

1. Do not repeat yourself
2. Every minute must have a unique tactical focus
3. Include real macro events:
 - Scuttle around 3:30

- First Dragon around 5:00–6:00
 - Rift Herald timing
 - Turret plates falling at 14:00
4. After minute 14, transition into rotations and objective control
 5. If enemy jungler is unknown, threat level must be Medium or High and warding must be emphasized

Why It Works

This transformed the output from generic lane advice into realistic ranked gameplay planning and made the report feel like Challenger-level coaching rather than basic chatbot suggestions.

Step 5 — Final Report Formatting

Purpose

The final LLM step receives the full populated state dictionary and converts it into a polished Markdown report.

Required sections include:

- Matchup Overview
- Power Spikes
- Recommended Build
- Tactical Timeline Table
- Win Condition
- Data Source Notes

Why this prompt is separate

Formatting should not be mixed with reasoning. If formatting is requested too early, the model often weakens the analysis quality. Separating the presentation layer ensures stronger reasoning quality and cleaner final outputs.

Final Reflection on Prompt Design

The biggest lesson from this project was that prompt design controls output quality more than model size.

The original pipeline was technically correct but strategically weak because prompts were too broad and under-constrained. Once prompts were redesigned around explicit reasoning boundaries and failure cases discovered during testing, output quality improved dramatically.

The most valuable prompt engineering principle learned was:

“Never trust the model to infer critical constraints implicitly.”

If a rule matters, it must be written explicitly.

This project demonstrated that strong multi-step agents are built through controlled reasoning design, not through a single powerful prompt.